CONTENTS

Advanced | Vector search infrastructure | Condensed | IBM Technology

# Vector Databases: Storing Meaning Instead of Fields

Understand embeddings, distance metrics and the approximate-nearest-neighbour indexes that make semantic search fast.

For engineers who need to choose, tune or debug a vector store rather than just call one · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video | Read the full lesson | Read the highlights

BEFORE YOU START

- What a relational database does
- Comfort reading Python
- Knowing what a vector is — a list of numbers with a position in space

---

## Overview

A relational database can store an image perfectly well — the binary blob, the file format, the creation date, whatever tags somebody typed in. What it cannot do is answer 'find me pictures with a similar colour palette' or 'find landscapes with mountains in the background', because those concepts are not in any column. That mismatch between how computers store data and how humans understand it is called the semantic gap.

Vector databases close it by storing meaning as geometry. Every item becomes an array of numbers positioned so that similar things sit close together, and 'find similar' becomes a mathematical operation over those positions rather than a predicate over fields.

The engineering interest is mostly in what happens after that idea. Comparing a query against millions of thousand-dimensional vectors exhaustively is too slow, so real systems use approximate nearest-neighbour indexes that trade a small amount of accuracy for a very large amount of speed. Understanding how HNSW and IVF make that trade — and what you give up — is the difference between using a vector database and being able to fix one.

KEY TAKEAWAYS

- ✓ The semantic gap is the mismatch between structured storage and conceptual similarity — SQL predicates cannot express 'looks like this'.
- ✓ An embedding is an array of numbers positioned so that semantically similar items land close together and dissimilar ones land far apart.
- ✓ Individual dimensions in real embeddings are almost never interpretable; the geometry as a whole carries the meaning.
- ✓ Each modality has its own embedding models, and vectors from different models are never comparable.
- ✓ Embedding models extract progressively more abstract features layer by layer — edges to objects, words to meaning.
- ✓ Exhaustive search is O(N·d) and becomes impossible at scale, which is why vector indexes exist.
- ✓ HNSW navigates a multi-layer proximity graph; IVF partitions the space into clusters and searches only the nearest few.
- ✓ ANN indexes trade a small, measurable amount of recall for very large speed gains — and that recall must actually be measured.

## 01 The semantic gap

0:00

> You can store an image in a relational database along with its metadata and tags, and still be unable to ask the questions that matter about it.

KEY POINTS

- → Relational storage handles the file and its metadata but not its content.
- → Manual tags fix the queryable attributes before anyone knows what will be asked.
- → Similarity is continuous; tags are discrete, so degree cannot be expressed.
- → Tagging is incomplete and inconsistent at any realistic scale.
- → The semantic gap is structural — it cannot be closed by tagging harder.

### What the relational row actually holds

Everything here is about the file rather than the picture. Nothing in the row would let you find this image by starting from a different image that looks like it.

```
CREATE TABLE images (
    id          SERIAL PRIMARY KEY,
    data        BYTEA,          -- the actual pixels, opaque to the database
    format      TEXT,           -- 'jpeg'
    created_at  TIMESTAMP,
    tags        TEXT[]          -- {'sunset','landscape','orange'}
);

-- answerable:   SELECT * FROM images WHERE 'sunset' = ANY(tags);
-- not answerable: "images with a similar colour palette to image 42"
--                 "landscapes with mountains in the background"
--                 "something with this mood"
```

## 02 Vectors as the representation

2:11

> Represent each item as an array of numbers positioned so that similar items are near one another, and similarity search becomes arithmetic.

KEY POINTS

- → An embedding is an array of numbers whose position encodes semantic content.
- → Similar items sit close together; dissimilar ones sit far apart.
- → 'What is similar?' becomes 'what is nearby?', a well-defined geometric query.
- → Images, text and audio all become vectors and share the same machinery.
- → The distance metric matters: cosine ignores magnitude, Euclidean does not, dot product mixes both.
- → Using a metric the embedding model was not trained for degrades results silently.

### The three metrics, and when each applies

If your vectors are L2-normalised, cosine and dot product rank identically and Euclidean produces the same ordering too — which is why many stores normalise on insert and stop worrying about it. If they are not normalised, the three can disagree substantially.

```
import numpy as np

def cosine(a, b):      # angle only — magnitude ignored
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def dot(a, b):         # angle and magnitude together
    return np.dot(a, b)

def euclidean(a, b):   # straight-line distance; smaller is closer
    return np.linalg.norm(a - b)


# after L2 normalisation all three rank identically:
def normalise(v):
    return v / np.linalg.norm(v)

# check what your embedding model was trained for before choosing —
# the wrong metric produces plausible, quietly worse results
```

## 03 What a vector actually encodes

3:38

> Each position represents a learned feature. In a toy example those features are readable; in real systems they are not.

KEY POINTS

- → Each position in the vector corresponds to a learned feature.
- → Real embeddings have hundreds or thousands of dimensions.
- → Individual dimensions are almost never interpretable — meaning is distributed.
- → You cannot filter or debug by inspecting single coordinates; only whole-vector comparisons are meaningful.
- → Anything you need to filter on must be stored explicitly as metadata.
- → Dimensionality trades expressiveness against memory, bandwidth and query time.

### The toy vector versus the real one

The left column is a teaching device. The right is what you will actually get back, and no coordinate in it has a name.

```
toy (interpretable, 3 dims)        real (1536 dims, uninterpretable)
──────────────────────────────────────────────────────────────────────

0.91  elevation change              [-0.0231,  0.0417, -0.0088,
0.15  urban elements                  0.0192, -0.0364,  0.0521,
0.83  warm colour                     0.0077,  0.0143, -0.0296,
                                      ... 1527 more ... ]

# no dimension means "elevation". the concept is spread across many
# coordinates, and no single one can be read, thresholded or filtered on.
```

## 04 Comparing two vectors

4:20

> Two sunsets, one mountain and one beach: similar where they share meaning, different where they do not.

KEY POINTS

- → Two items can be alike in some dimensions and unlike in others simultaneously.
- → The distance metric aggregates per-dimension agreement into one score.
- → A tag system must call two items same or different; a vector expresses degree.
- → What counts as similar depends on where the query sits, not on stored labels.
- → This is why semantic search generalises to questions nobody anticipated.

### Two sunsets, dimension by dimension

Aggregate distance hides this structure — you see one number and not which dimensions produced it. Being able to picture the per-dimension view is what makes similarity results interpretable when they surprise you.

```
                  mountain   beach    agreement
──────────────────────────────────────────────────
elevation           0.91      0.12     far apart
urban elements      0.15      0.08     both low
warm colour         0.83      0.89     nearly equal

# similar overall — the sunset dominates — but clearly distinguishable
# on the axis that separates a mountain from a shoreline
```

## 05 Where embeddings come from

5:48

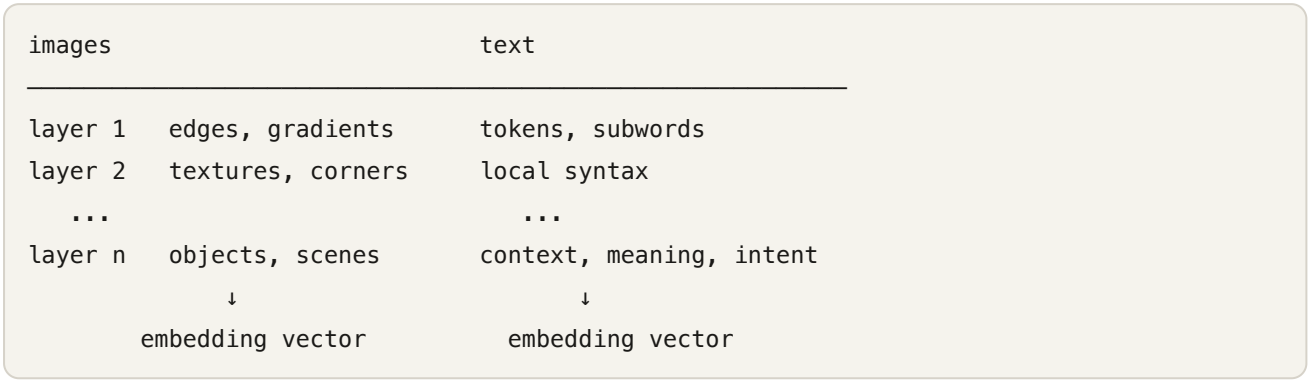
Embedding models trained on massive datasets, with each layer extracting progressively more abstract features.

KEY POINTS

- → Different modalities use different embedding models: CLIP for images, Wav2vec for audio, text encoders for text.
- → Layers extract progressively more abstract features — edges to objects, words to meaning.
- → The embedding is read from a deep layer that captures essential characteristics.
- → Static word embeddings give one vector per word; contextual encoders vary by surrounding text.
- → Vectors are only comparable within the same model — mixing models produces meaningless results.
- → Changing embedding model requires re-embedding the entire corpus.

### Abstraction increasing with depth

The reason the embedding comes from a deep layer rather than an early one. Early activations describe the surface of the input; deep activations describe what it is.




## 06 Similarity search, and why brute force runs out

7:16

> Comparing a query against every vector is exact and simple, and becomes impossible somewhere in the millions.

KEY POINTS

- → Exhaustive search is exact and simple, and correct for small collections.
- → Its cost is O(N·d) per query — every vector, every dimension.
- → At millions of vectors it is bounded by memory bandwidth as well as arithmetic.
- → ANN indexes give up exactness for orders-of-magnitude speed.
- → The trade is only sound if the lost recall is actually measured.

### Exhaustive search, and where it stops working

The vectorised form is much faster than a Python loop and still linear in the corpus. Somewhere between a hundred thousand and a few million vectors — depending on dimensions, hardware and your latency budget — it stops meeting a real-time requirement.

```
import numpy as np

def exact_search(query, matrix, k=10):
    # matrix: (N, d), L2-normalised so dot product == cosine
    scores = matrix @ query                 # (N,) — every vector, every dim
    top = np.argpartition(-scores, k)[:k]   # cheaper than a full sort
    return top[np.argsort(-scores[top])]

# 1M vectors x 1536 dims x 4 bytes = 6.1 GB read per query
# correctness: perfect.   latency: not interactive.
```

## 07 ANN indexing: HNSW and IVF

8:00

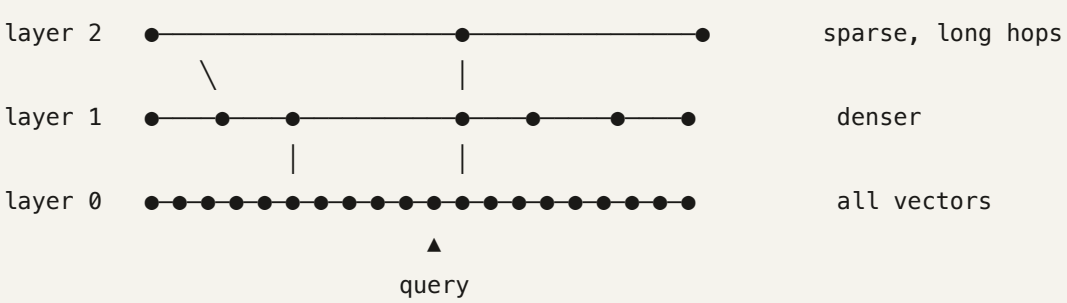
> Two ways to avoid looking at everything — navigate a proximity graph, or search only the nearest clusters.

KEY POINTS

- → HNSW builds a layered proximity graph and greedily navigates from sparse to dense layers.
- → HNSW parameters: M for connectivity and memory, ef_construction for build quality, ef_search for the recall/latency dial at query time.
- → IVF clusters the space with k-means and searches only the nearest nprobe clusters.
- → IVF needs a training pass and degrades if the data distribution shifts afterwards.
- → HNSW gives better recall per millisecond and costs more memory; IVF is leaner and quantises well.
- → Both trade a small amount of accuracy for large speed gains.

### How an HNSW search moves

The upper layers exist purely to cover distance quickly. Without them the greedy walk on the dense bottom layer would take many more hops and could get stuck in a local minimum far from the query.




```
# enter at the top, greedily step toward the query, descend when no
# neighbour improves, repeat. the bottom layer does the fine work.
```

## 08 Vector databases inside RAG

8:41

> Store document chunks as embeddings, find the relevant ones by similarity, hand them to a language model.

KEY POINTS

- → In RAG the vector store holds chunks and their embeddings for retrieval.
- → Chunk size decides what a single vector must represent.
- → The embedding model defines similarity and cannot change without a full re-index.
- → The distance metric must match how the embedding model was trained.
- → ANN recall is a hard ceiling on the whole application's accuracy.
- → Debug RAG failures at the retrieval layer before touching the prompt.

### The full path, and the knob at each stage

Six stages, and five of them are decisions that silently cap what the model can do. Only the last is visible in a prompt, which is why it receives most of the attention and deserves less of it.

```
document ──chunk──> passages ──embed──> vectors ──index──> ANN structure
            |                   |                   |
         size,               which model,      M, ef_search,
         overlap,            which metric      nprobe, recall
         boundaries

question ──embed──> vector ──search──> top-k chunks ──> prompt ──> answer
              |                  |                          |
        same model as       k, filters              the only part
        the corpus                                  most teams tune
```

## Glossary

### Semantic gap

The mismatch between how computers store data — files, fields, tags — and how humans understand its meaning.

### Vector embedding

An array of numbers representing an item, positioned so that semantically similar items land close together.

### Vector database

A store built to hold embeddings and retrieve them by similarity, with the indexing and filtering machinery that makes that fast.

### Cosine similarity

The cosine of the angle between two vectors. Ignores magnitude, which is usually what you want for text.

### Euclidean distance

Straight-line distance between two points in the vector space. Sensitive to magnitude as well as direction.

### Dot product

A similarity measure combining alignment and magnitude. Equivalent to cosine when vectors are normalised.

### L2 normalisation

Scaling a vector to unit length, which makes cosine, dot product and Euclidean produce identical rankings.

### Approximate nearest neighbour (ANN)

Algorithms that find vectors very likely to be the closest, trading a little accuracy for a large speed gain.

### HNSW

Hierarchical Navigable Small World: a multi-layer proximity graph navigated greedily from sparse upper layers down to the dense base.

### IVF

Inverted File Index: partitions the space into clusters and searches only the nearest few, controlled by nprobe.

### ef_search

HNSW's query-time beam width. The main dial for trading recall against latency, changeable without rebuilding the index.

### nprobe

How many IVF clusters a query searches. One is fastest and least accurate; all of them is equivalent to exhaustive search.

### Quantisation

Compressing vectors to fewer bits per dimension to cut memory, at some cost in precision. Often paired with IVF at large scale.

### Recall@k

The fraction of true nearest neighbours an approximate index actually returns. The ceiling on any system built on it.

### Static vs contextual embedding

Static models give each word one fixed vector; contextual encoders produce representations that depend on the surrounding text.

## Check yourself

Answer first, then open each one.

### Why can't you close the semantic gap by simply adding more tags to your relational schema?

Because tags are discrete, unary properties chosen in advance, while similarity is continuous and relational. There is no way to express 'somewhat sunset' or 'close to image 42' as a column value, and nobody tags for an attribute until after someone needs it. The limitation is structural, not a matter of effort.

### Your embedding model was trained with cosine similarity, but your index is configured for Euclidean distance and the vectors are not normalised. What happens?

Nothing visibly. Queries succeed and results come back ranked, but the ordering is influenced by vector magnitude — which the model never intended to carry meaning — so relevance quietly degrades. The fix is to L2-normalise on insert, after which all three metrics rank identically anyway.

### Why is it impossible to build a filter by thresholding a single dimension of a real embedding?

Because meaning is distributed across the whole vector rather than allocated one concept per axis. No coordinate corresponds to 'elevation' or any other nameable property, so a threshold on it selects nothing coherent. Anything you need to filter on must be extracted explicitly and stored as metadata.

### What do HNSW's upper layers actually contribute, given that every vector lives in the bottom layer?

They cover long distances in few hops. Starting a greedy walk directly on the dense bottom layer would take many more steps and risks settling in a local minimum far from the query. The sparse upper layers act as an express network that gets the search into the right neighbourhood before the fine-grained work begins.

### Why does an IVF index with nprobe=1 miss neighbours that are extremely close to the query?

Because it searches only the single nearest cluster, and a vector sitting just across a cluster boundary is never examined regardless of its actual distance. Cluster membership is decided at index time by proximity to a centroid, not by proximity to any particular query. Raising nprobe searches neighbouring cells and recovers them.

### A RAG system gives a wrong answer. Why should you measure ANN recall before rewriting the prompt?

Because recall is a hard ceiling sitting below everything visible. If the index returns 0.85 of true nearest neighbours, then roughly fifteen percent of the time the best chunk never reaches the model at all, and no prompt can compensate for text that was never supplied. Measure against exact search on the same vectors, then work upward.

## Where to go next

- → Build an exact search over 100,000 of your own vectors and use it as ground truth, then measure your production index's recall@10 at its current settings — most teams have never seen this number.
- → Sweep ef_search from 16 to 512 and plot recall against p95 latency; the knee of that curve is your actual operating point, and it is rarely where the default sits.
- → Compare HNSW and IVF on the same data at matched recall and compare memory footprint — the gap decides which is affordable at your scale.
- → Take a query with a selective metadata filter and compare post-filtering, over-fetching and native filtered search; the result counts alone are usually enough to settle the design.
- → Re-embed a small corpus with a different model and confirm for yourself that cross-model queries return confident nonsense — the failure is much more memorable once seen.
- → Try product quantisation on a million-vector index and measure the three-way trade between memory, recall and latency.

→ Read the HNSW paper (Malkov & Yashunin, 2016) for why a hierarchy of proximity graphs gives logarithmic-ish search behaviour.

---